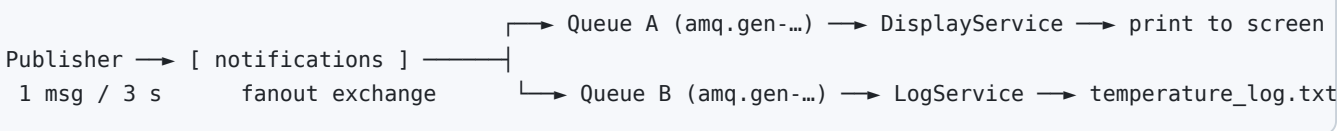
Task 2 — Temperature Sensor Notification (RabbitMQ)

Name	Xiao Ma
Student ID	6730613037
Major	Software Engineering
Git Repository	https://github.com/orimaxmaxiao-coder/rabbitmq-temperature
Date	October 25, 2026

1. Requirements and Completion

Requirement	Status
Create a fanout exchange called `notifications`	✅ Done (see Screenshots 6 and 7)
Publisher sends a sensor message `{"sensor_id","temperature","timestamp"}`	✅ `publisher.js`
Temperature is a random number; timestamp uses `toISOString()`	✅ Random 20.0–40.0 °C, `new Date().toISOString()`
Publisher uses `setInterval` to send 1 message every 3 seconds	✅ `setInterval(..., 3000)`
DisplayService: print the temperature to the screen	✅ `display_service.js` (Screenshot 4)
LogService: save the temperature to a text file	✅ `log_service.js` → `temperature_log.txt` (Screenshot 5)
Create a Git repository and submit the link as a PDF to Google Drive	✅ Pushed to GitHub (Screenshots 8 and 9); this PDF is the submission

2. How It Works



A **fanout** exchange ignores the routing key and **copies every message** to all queues bound to it. When each subscriber starts, it creates a temporary, exclusive queue (RabbitMQ generates a name such as `amq.gen-xxxx`) and binds it to `notifications`. Because of this, both services receive every message.

Screenshot 7 shows that **Publish (In)** is about half of **Publish (Out)**: one message comes in and two copies go out. This is the fanout behavior.

3. What Each Plugin / Dependency / Tool Means

The software, libraries, and tools used in this project, listed in the order they appear in the workflow:

Name	What it is	Role in this project
RabbitMQ	An open-source **message broker** that passes messages between programs using the AMQP protocol.	The core server. It holds the `notifications` exchange and the two queues, and forwards the Publisher's messages to the subscribers.
Erlang	RabbitMQ is written in Erlang, so it needs the Erlang runtime.	Already included in the Docker image (Screenshot 6 shows Erlang 26.2.5.16), so no separate installation is needed.
rabbitmq_management (management plugin)	An **official plugin** shipped with RabbitMQ that provides a web dashboard and an HTTP API.	Open `http://localhost:15672` (user/password guest/guest) to see exchanges, queues, bindings, and message rates (Screenshots 6 and 7). The `rabbitmq:3-management` image enables it by default.
Docker / Docker Desktop	A **container** platform that packages software and its environment into an "image" so it runs with a single command, with no manual installation.	Runs the RabbitMQ server, so Erlang and RabbitMQ do not have to be installed on Windows (Screenshot 2).
Docker Compose `docker-compose.yml`	A YAML file that describes which containers to start and which ports to expose.	Defines the `rabbitmq` service: image `rabbitmq:3-management`, port **5672** (used by the programs) and port **15672** (web dashboard). Command: `docker compose up -d`.
Node.js	A runtime for running JavaScript on a computer.	Runs the three programs `publisher.js`, `display_service.js`, and `log_service.js` (v24.22.0 on this machine).
npm `package.json`	The **package manager** bundled with Node.js. `package.json` records the project's dependencies and shortcut commands.	`npm install` downloads the dependencies. `npm run publisher / display / log` are shortcuts for starting the three programs.

amqplib	An **AMQP client library** for Node.js that lets JavaScript programs connect to RabbitMQ.	Used by all three programs: `amqp.connect()` to connect, `assertExchange()` to create the exchange, `publish()` to send, and `assertQueue()/bindQueue()/consume()` to receive.
package-lock.json	Generated automatically by npm; it locks the exact versions of the dependencies.	Ensures that anyone who runs `npm install` gets the same version of amqplib.
node_modules/	The folder where npm stores the downloaded dependency code.	It is large and can be regenerated at any time, so it is listed in `.gitignore` and not uploaded to GitHub.
fs / path	**Built-in** Node.js modules that need no installation. `fs` reads and writes files; `path` builds file paths.	LogService uses `fs.appendFileSync()` to append each temperature reading to `temperature_log.txt`.
Git	A **version control** tool that records every change to the code as a commit.	The local repository has 3 commits: initial code → bilingual console output → `.gitignore` update.
.gitignore	Tells Git which files not to track or upload.	Excludes `node_modules/` and the helper scripts (check_env.bat, setup.bat, push.bat, etc.).
GitHub	A website that hosts Git repositories online.	Hosts the assignment code and provides the repository link for submission (Screenshot 9).
Git Credential Manager	A sign-in helper bundled with Git for Windows.	On the first `git push`, it opens the browser to sign in to GitHub (the "please complete authentication in your browser" line in Screenshot 8).
.bat batch scripts	Windows automation scripts that run a series of commands when double-clicked.	`setup.bat`: start Docker and RabbitMQ, run npm install, make the Git commit. `run.bat`: open the three service windows at once. `push.bat`: commit and push to GitHub. They are conveniences only, not requirements.

4. Project Files

File	Purpose
`publisher.js`	Publisher: connects to RabbitMQ → creates the fanout exchange `notifications` → sends `{sensor_id:"s01", temperature: random, timestamp: now}` every 3 seconds.
`display_service.js`	Subscriber 1: creates a temporary queue, binds it to `notifications`, and prints each message it receives.
`log_service.js`	Subscriber 2: binds the same way and appends each message to `temperature_log.txt` as `timestamp,sensor,temperature`.
`docker-compose.yml`	Starts the RabbitMQ container (with the management dashboard).
`package.json` / `package-lock.json`	Dependency (amqplib) and npm shortcut commands.
`README.md`	Bilingual (English/Chinese) project description.
`run.bat`	Starts all three services on Windows with one double-click.
`temperature_log.txt`	Temperature records written by LogService (the run output).

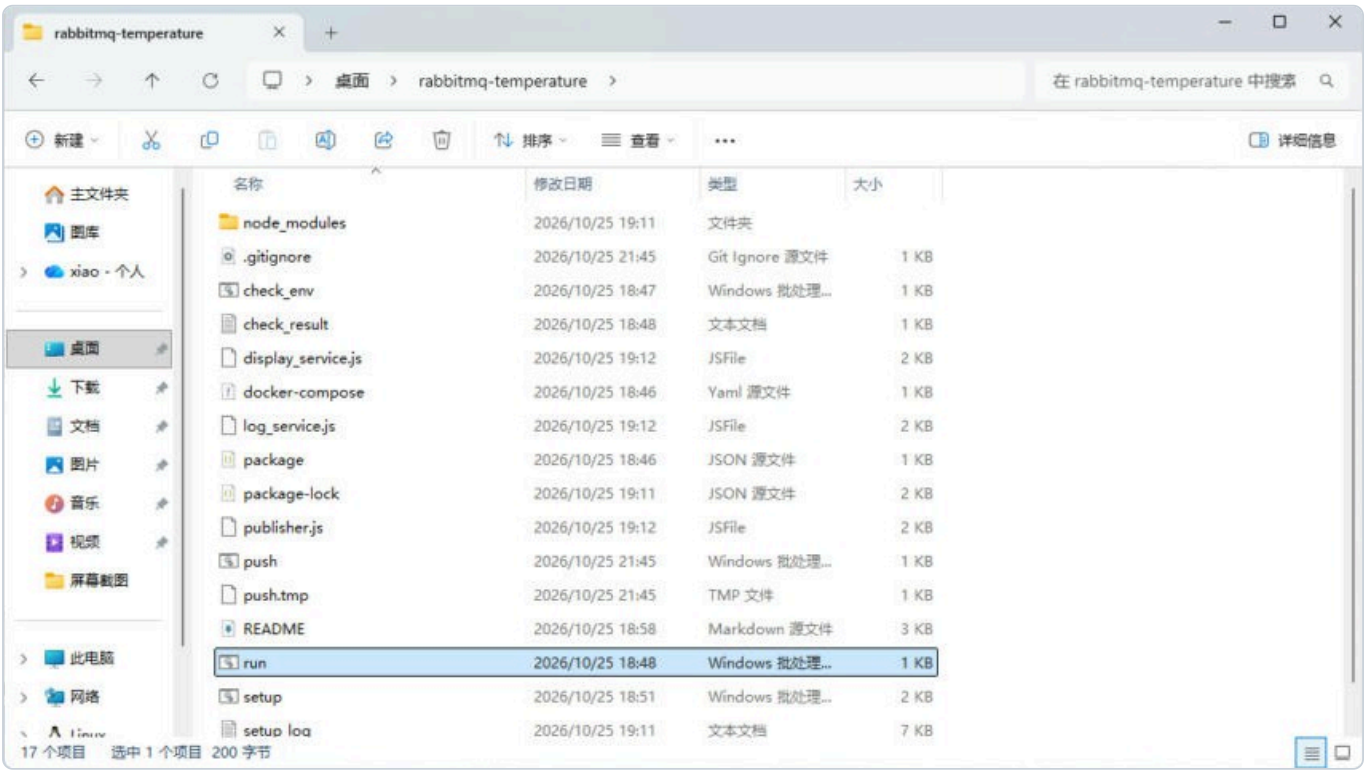
Key Code

```
// publisher.js — create the fanout exchange and send every 3 seconds
await channel.assertExchange('notifications', 'fanout', { durable: false });
setInterval(() => {
  const message = {
    sensor_id: 's01',
    temperature: Number((20 + Math.random() * 20).toFixed(1)),
    timestamp: new Date().toISOString(),
  };
  channel.publish('notifications', '', Buffer.from(JSON.stringify(message)));
}, 3000);

// display_service.js / log_service.js — bind a temporary queue to the exchange
const { queue } = await channel.assertQueue('', { exclusive: true });
await channel.bindQueue(queue, 'notifications', '');
channel.consume(queue, (msg) => { /* print or write to file */ }, { noAck: true });
```

5. Step-by-Step Screenshots

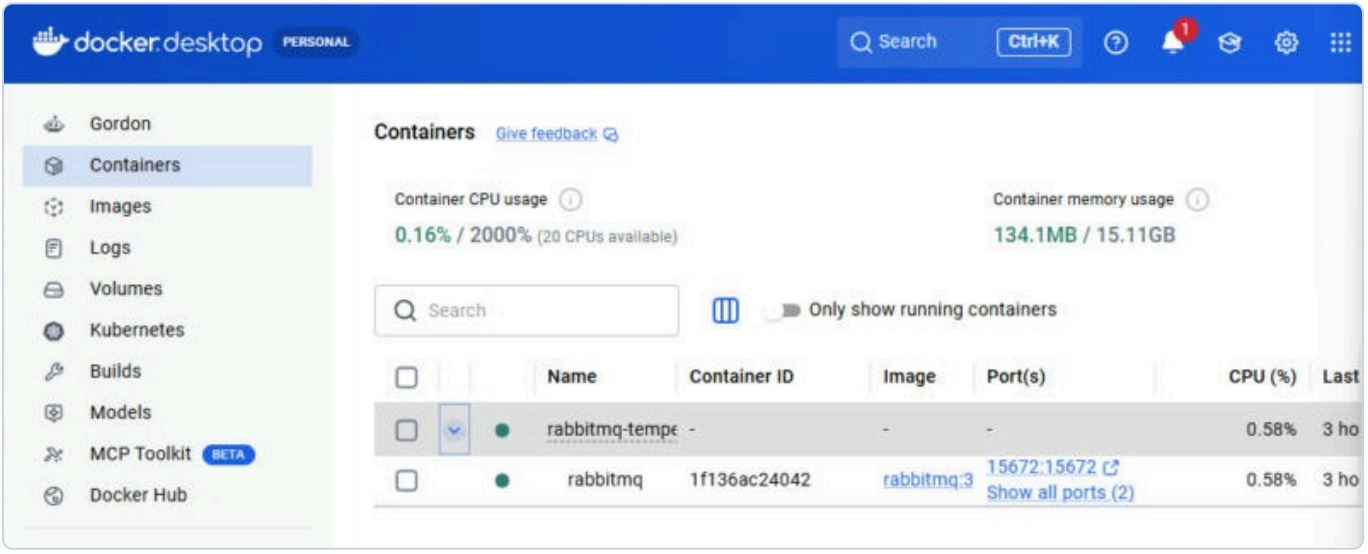
Step 1 Project Folder



Screenshot 1: the rabbitmq-temperature folder on the Desktop (node_modules and package-lock.json appear after npm install)

Step 2 Start RabbitMQ (Docker Desktop)

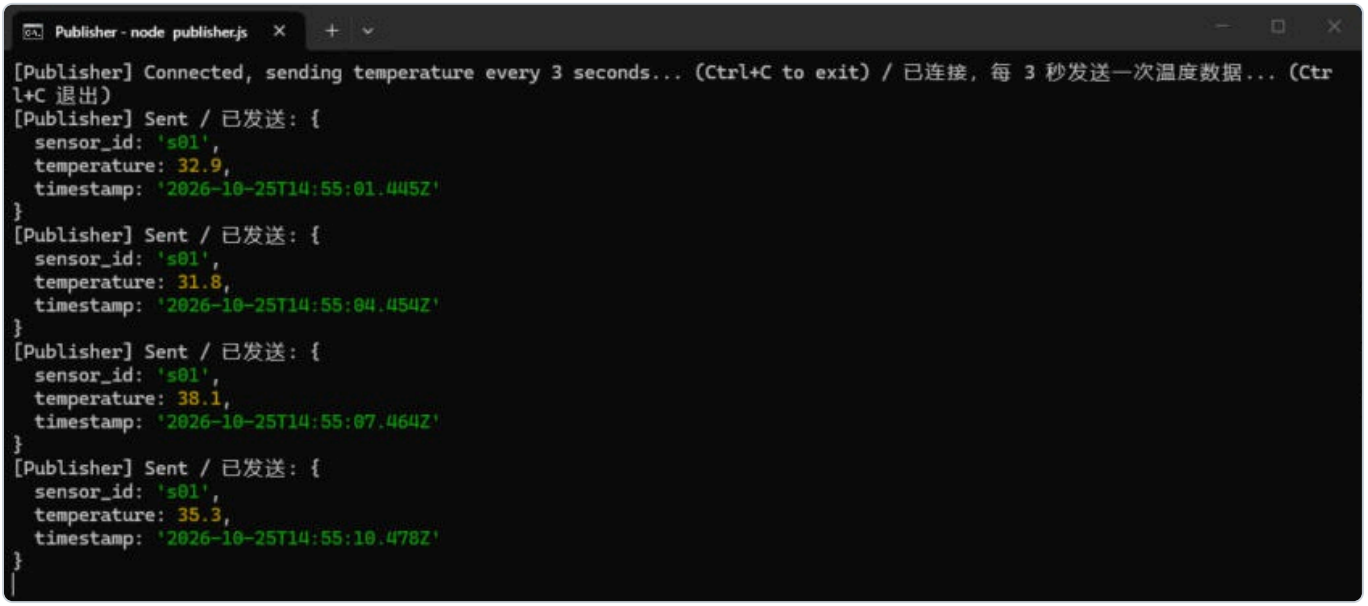
Command: `docker compose up -d`. Docker Desktop shows **Engine running** in the bottom-left corner, the **rabbitmq** container is green (running), and port 15672 is exposed.



Screenshot 2: the rabbitmq container running in Docker Desktop

Step 3 Run the Publisher (one message every 3 seconds)

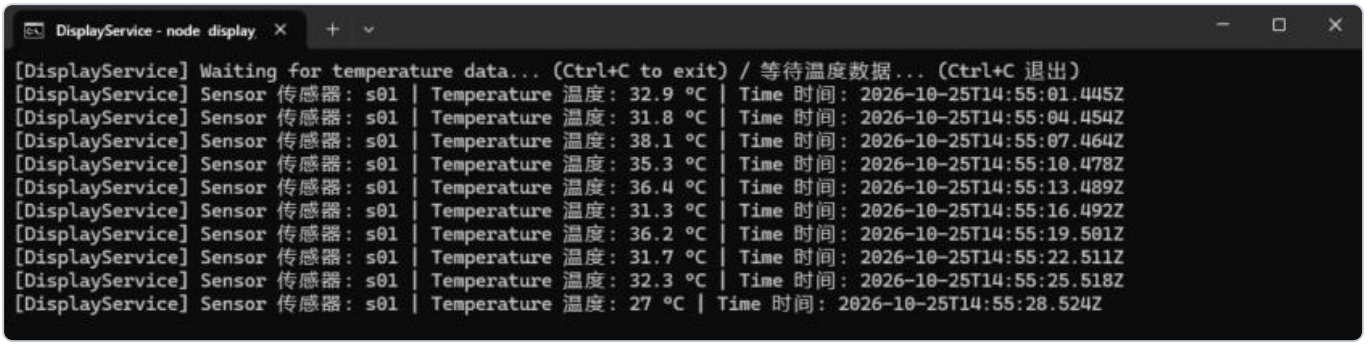
Command: `npm run publisher` (or `node publisher.js`). The timestamps are 3 seconds apart and the temperatures are random.



Screenshot 3: sensor messages sent by the Publisher

Step 4 DisplayService — Print Temperatures to the Screen

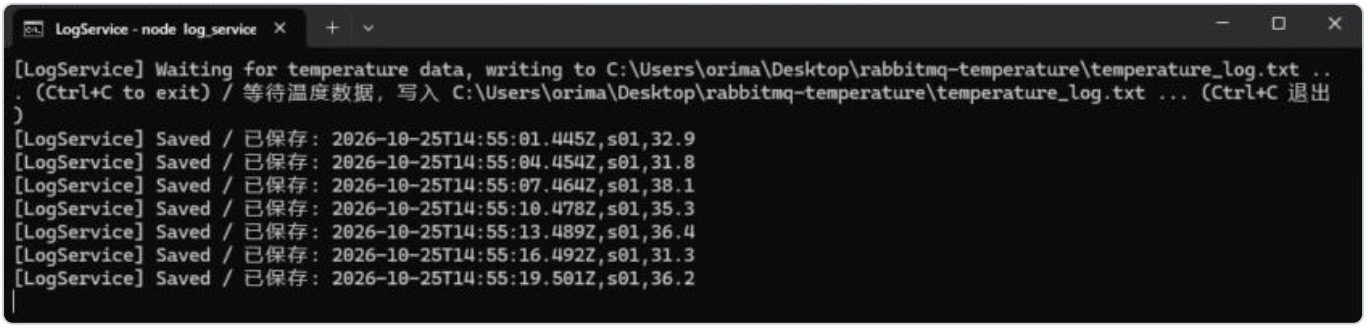
Command: `npm run display` (or `node display_service.js`).



Screenshot 4: DisplayService receives and prints every temperature reading

Step 5 LogService — Save Temperatures to a Text File

Command: `npm run log` (or `node log_service.js`). The timestamps match Screenshot 4 exactly, which shows that both subscribers receive **the same messages**.



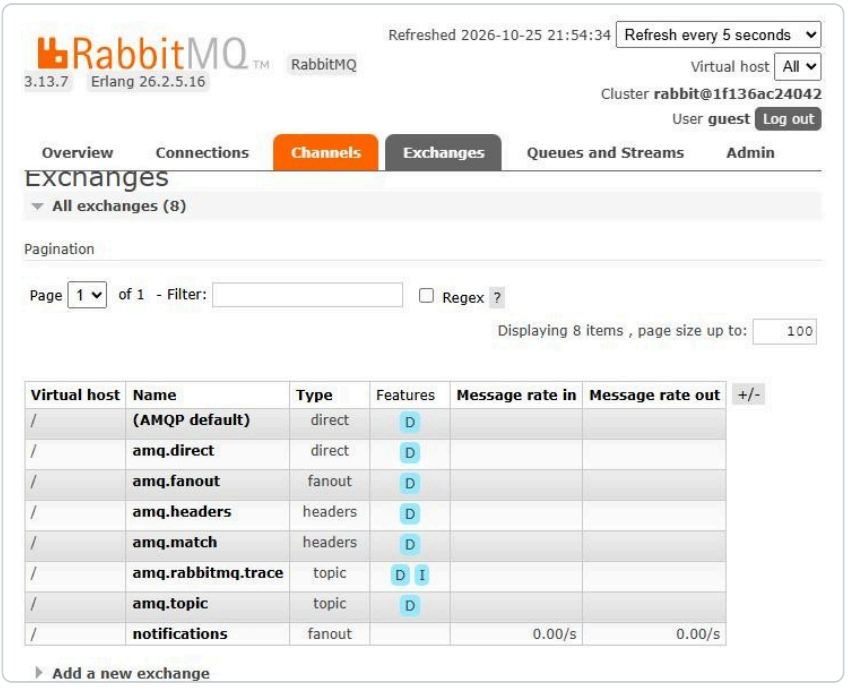
Screenshot 5: LogService writing to temperature_log.txt

Excerpt from `temperature_log.txt` (74 records in total):

```
2026-10-25T14:57:01.782Z,s01,32.8
2026-10-25T14:57:04.786Z,s01,39
2026-10-25T14:57:07.799Z,s01,39.5
2026-10-25T14:57:10.804Z,s01,21.5
2026-10-25T14:57:13.818Z,s01,23.7
2026-10-25T14:57:16.827Z,s01,39.1
```

Step 6 RabbitMQ Management Dashboard — the notifications Exchange

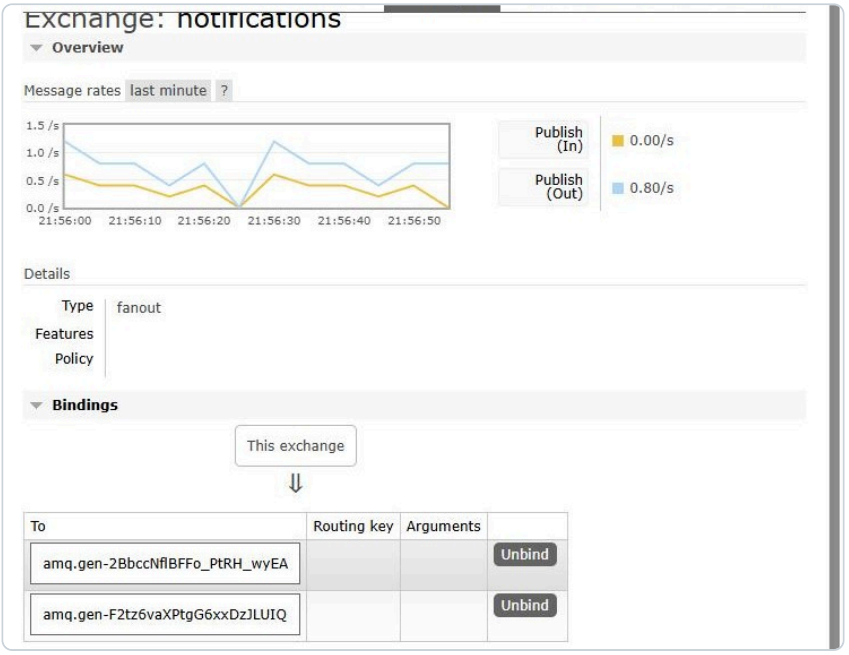
Open `http://localhost:15672` in a browser, log in with guest / guest, and click **Exchanges**. The last row is **notifications**, of type **fanout**.



Screenshot 6: list of exchanges

Step 7 Two Queues Bound to the Exchange

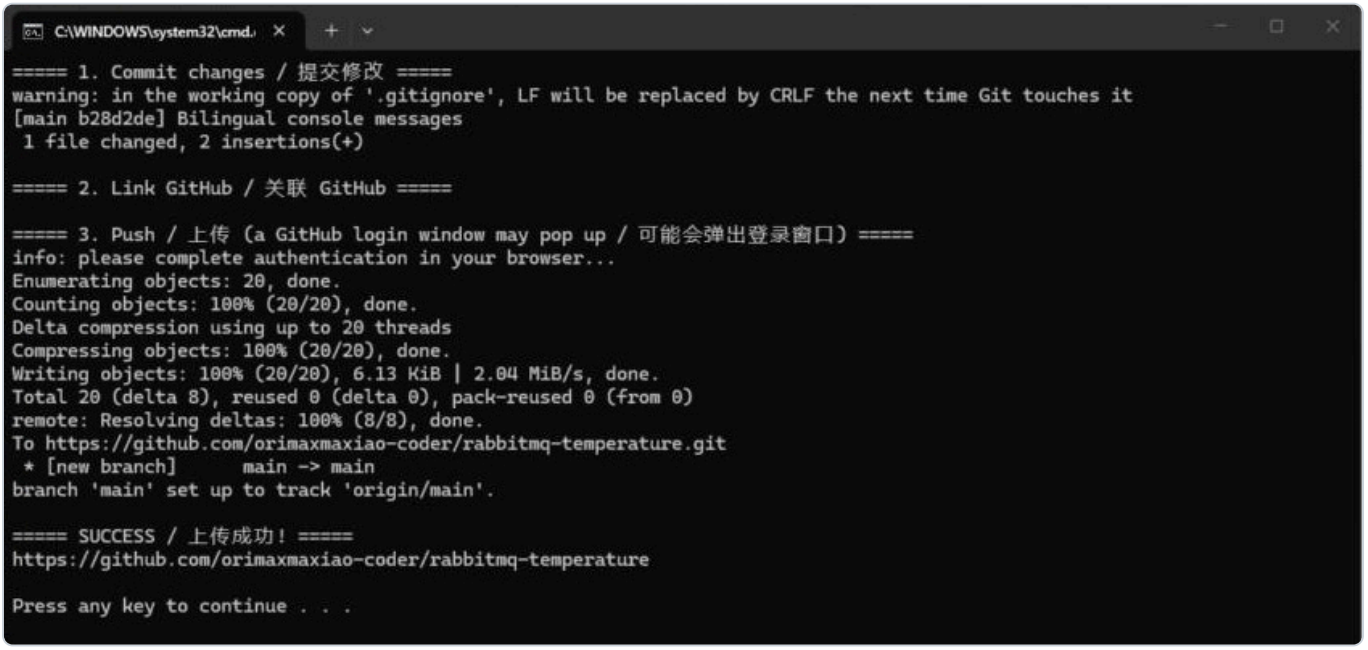
Open notifications → **Bindings**: two auto-named queues are bound (one for DisplayService, one for LogService). The message rate Out ≈ 2 × In, which proves each message is broadcast to both subscribers.



Screenshot 7: message rates and bindings of notifications

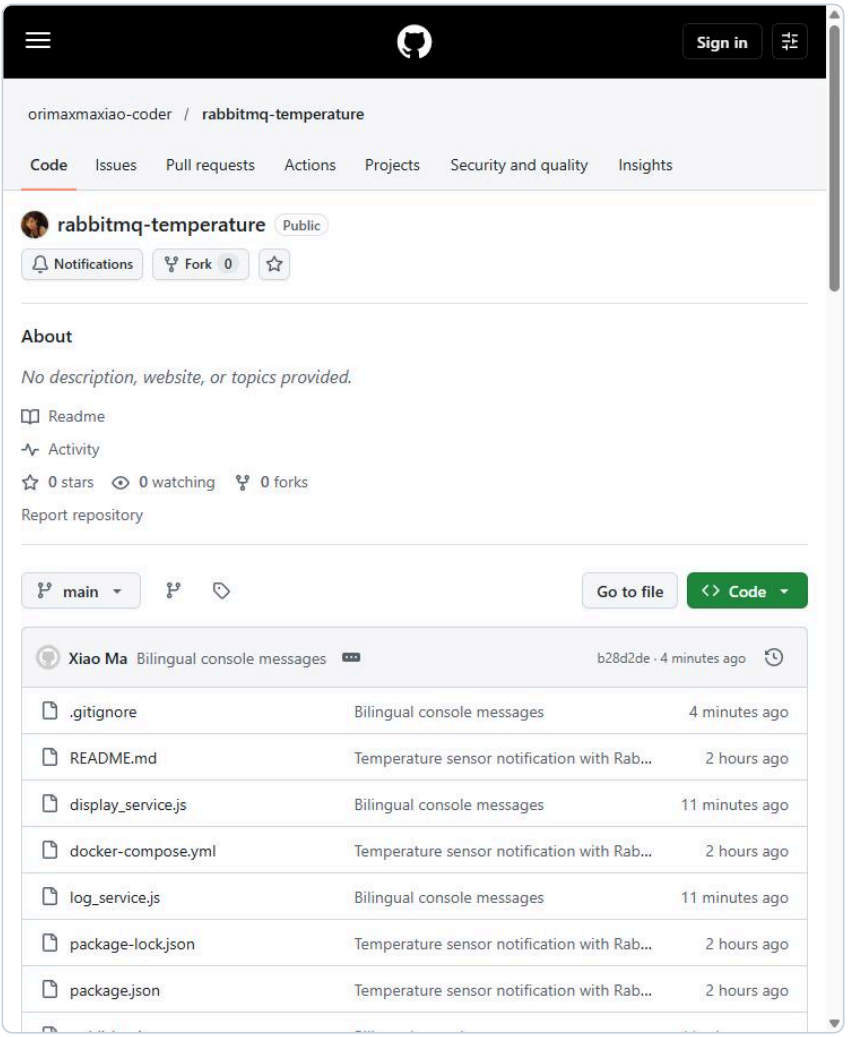
Step 8 Commit and Push to GitHub

```
git add .
git commit -m "Bilingual console messages"
git remote add origin https://github.com/orimaxmaxiao-coder/rabbitmq-temperature.git
git push -u origin main
```



Screenshot 8: git push succeeded (main -> main)

Step 9 GitHub Repository



Screenshot 9: https://github.com/orimaxmaxiao-coder/rabbitmq-temperature

6. How to Run Without the .bat Scripts

The three `.bat` files (`setup.bat`, `run.bat`, `push.bat`) only bundle commands so they can be run with a double-click. They are not required. Without them, open a terminal in the project folder (right-click inside the folder → **Open in Terminal**) and type the commands below.

6.1 Instead of setup.bat — prepare the environment (first time only)

Open **Docker Desktop** first and wait until it shows **Engine running**. Then:

```
# 1. Start the RabbitMQ container
docker compose up -d

# 2. Install the dependency (amqplib)
npm install

# 3. Create the Git repository and make the first commit
git init
git config user.name "Xiao Ma"
git config user.email "your-email@example.com"
git add .
git commit -m "Temperature sensor notification with RabbitMQ fanout exchange"
git branch -M main
```

Check that RabbitMQ is running: `docker ps` should list a container named **rabbitmq**.

6.2 Instead of run.bat — start the three services

Open **three** terminal windows in the project folder and run one command in each. Start the two subscribers first, then the publisher, so no messages are missed:

```
# Terminal 1 — DisplayService
node display_service.js

# Terminal 2 — LogService
node log_service.js

# Terminal 3 — Publisher
node publisher.js
```

The npm shortcuts do the same thing: `npm run display`, `npm run log`, `npm run publisher`.

To stop, press **Ctrl + C** in each window. To stop RabbitMQ as well: `docker compose down`.

6.3 Instead of push.bat — upload to GitHub

```
# 1. Commit the latest changes
git add .
git commit -m "Update"

# 2. Link the GitHub repository (first time only)
git remote add origin https://github.com/orimaxmaxiao-coder/rabbitmq-temperature.git

# 3. Upload
git push -u origin main
```

After the first push, later uploads only need `git add .`, `git commit -m "..."` and `git push`.

6.4 Summary

Script	What it does	Manual equivalent
`setup.bat`	Starts Docker and RabbitMQ, installs dependencies, makes the first Git commit	`docker compose up -d` → `npm install` → `git init` / `git add .` / `git commit`
`run.bat`	Opens the three service windows	`node display_service.js`, `node log_service.js`, `node publisher.js` in three terminals
`push.bat`	Commits and pushes to GitHub	`git add .` → `git commit` → `git remote add origin …` → `git push -u origin main`

7. Submission

> **Git repository link:** https://github.com/orimaxmaxiao-coder/rabbitmq-temperature
> This PDF is submitted to the instructor's Google Drive folder.